

TIME AND SPACE COMPLEXITY

By Jorge Galvis

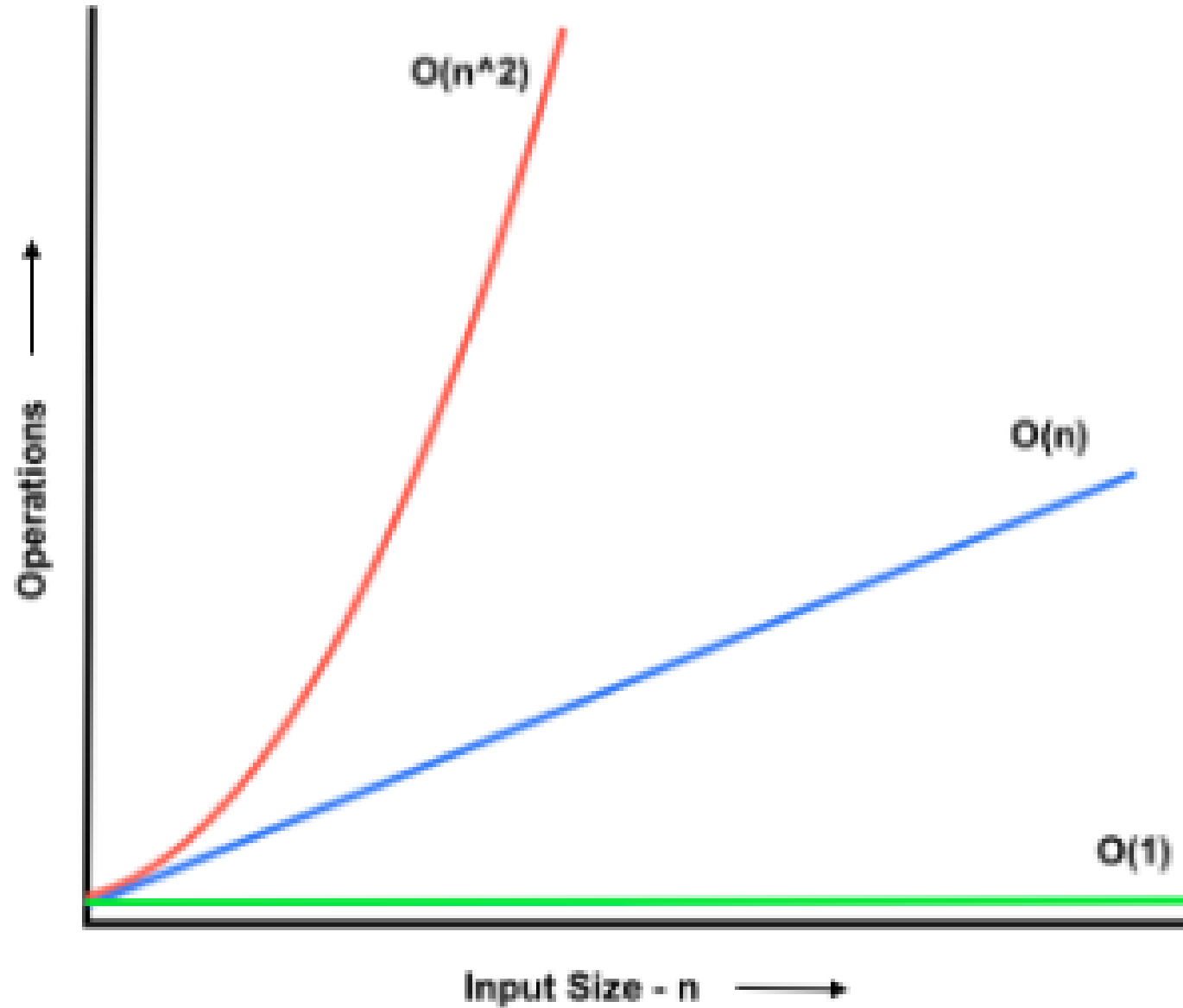
BIG O

- BigO measures how long an algorithm takes to run.
- We measure the speed of an algorithm based on the approximate number of steps that it takes to run.
- We calculate Big O in terms of the input's size.
- We judge speed based on the number of steps and not calculated time.
- We only care about the steps that increase based on the input size (additional space).
- For space complexity we care about the number of new variables.

BIG O

Taken from <https://skilled.dev/>

Big O Complexity



EXAMPLE

Let's normalize 1B operations in one second
and see how it behaves

Taken from <https://skilled.dev/>

Input size	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	0.003ns	0.01ns	0.033ns	0.1ns	1ns	3.65ms
20	0.004ns	0.02ns	0.086ns	0.4ns	1ms	77years
30	0.005ns	0.03ns	0.147ns	0.9ns	1sec	8.4×10^{15} years
40	0.005ns	0.04ns	0.213ns	1.6ns	18.3min	--
50	0.006ns	0.05ns	0.282ns	2.5ns	13days	--
100	0.07	0.1ns	0.644ns	0.10ns	4×10^{13} years	--
1,000	0.010ns	1.00ns	9.966ns	1ms	--	--
10,000	0.013ns	10ns	130ns	100ms	--	--
100,000	0.017ns	0.10ms	1.67ms	10sec	--	--
1'000,000	0.020ns	1ms	19.93ms	16.7min	--	--
10'000,000	0.023ns	0.01sec	0.23ms	1.16days	--	--
100'000,000	0.027ns	0.10sec	2.66sec	115.7days	--	--
1,000'000,000	0.030ns	1sec	29.90sec	31.7years	--	--

METHODOLOGY

Worst case

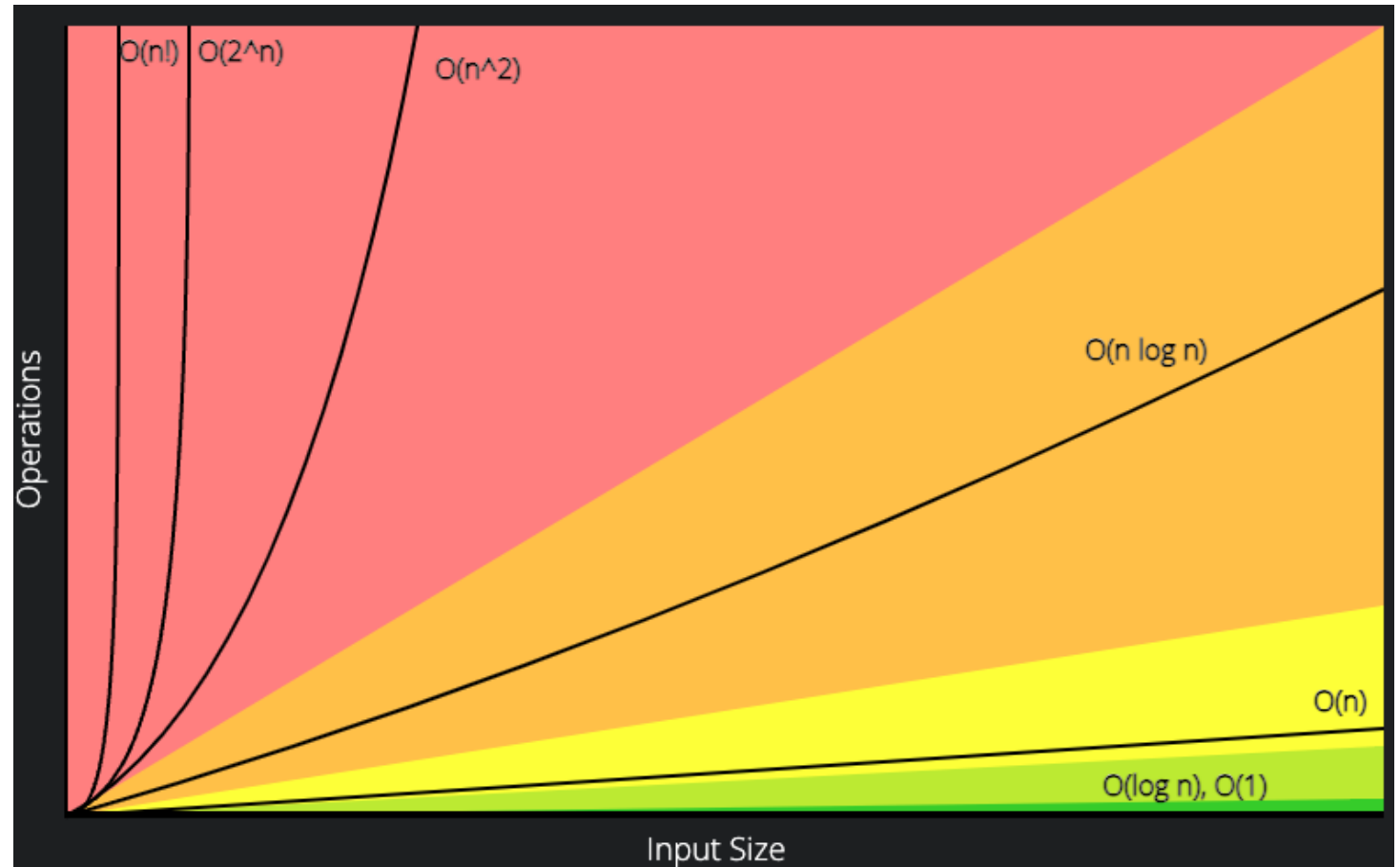
Drop constants

Drop less
significant terms

Account for all
input

COMMON BIG O'S

- Taken from <https://skilled.dev/>



EXAMPLE (IS UNIQUE STRING)

- Is Unique: Implement an algorithm to determine if a string has all unique characters.

SOLUTION $O(N^2)$

- $O(n^2)$ in time.
- $O(1)$ in space.

```
def is_unique_v1(word: str) -> bool:

    if len(word) > 127:
        return False

    if len(word) == 0:
        return True

    n: int = len(word)
    i: int = 0
    j: int = 0

    while i < n - 1:
        j = i + 1
        while j < n:
            if word[i] == word[j]:
                return False
            j += 1
        i += 1
    return True
```

SOLUTION $O(N)$

- $O(n)$ in time.
- $O(n)$ in space.

```
def is_unique_v2(word: str) -> bool:

    if len(word) > 127:
        return False

    if len(word) == 0:
        return True

    seen: dict = {}

    for char in word:
        try:
            if seen[char]:
                return False
        except KeyError:
            seen[char] = True

    return True
```

EXAMPLE (FIND THE MEDIAN)

- Solve <https://www.hackerrank.com/interview/preparation-kits/three-month-preparation-kit/three-month-week-one/challenges>

EXAMPLES

- $O(n \log n)$ in time.
- $O(1)$ in space.

```
def findMedian(arr):  
    # Write your code here  
    if not arr:  
        raise Exception('Empty arr')  
    arr = sorted(arr)  
    return arr[len(arr)//2]
```